

Reviewer Pack — Kronecker Coefficient Asymmetry in Permanent vs Determinant ...

Entry cb66523ac3ef · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 12:48:34 UTC

Kronecker Coefficient Asymmetry in Permanent vs Determinant Decompositions

Entry ID: cb66523ac3ef **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-11 12:48:34 UTC

1. Conjecture

Field A (mathematical branch): Schur-Weyl Duality **Field B** (complexity object): Homogeneous Polynomial Complexity

Statement:

For all $n \geq 2$, the number of distinct irreducible representations in the decomposition of perm_n under the symmetric group S_n exceeds the number for det_m where $m < n^{\{1.5\}}$, and this count is preserved under linear substitutions.

Rationale (proposer’s reasoning):

Schur-Weyl duality decomposes tensor powers into symmetric group irreps. Permanent’s decomposition has more distinct components than determinant’s due to its non-GL(n)-invariant structure, which could mirror complexity gaps via representation-theoretic invariants.

Taxonomy category: GCT_DET_PERM (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): dd8ae938acd5d5b8

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.95	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import combinations, permutations

def factorial(n):
    if n == 0:
        return 1
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

def hook_length_formula(partition):
    n = len(partition)
    numerator = factorial(factorial(n))
    denominator = 1
    for row in partition:
        for cell in range(len(row)):
            hook = row[cell] - cell + (n - row[cell])
            denominator *= hook
    return numerator // denominator

def generate_partition(n):
    if n == 0:
        return [[]]
    partitions = []
    for i in range(1, n + 1):
        for p in generate_partition(n - i):
            partitions.append([i] + p)
    return partitions

def count_irreducible_components(decompositions, n):
    counts = set()
    for decomposition in decompositions:
        for part in decomposition:
            if len(part) == n:
                counts.add(tuple(sorted(part)))
    return len(counts)
```

```

def run_trial(seed: int) -> dict:
    random.seed(seed)
    results = []
    for n in range(2, 41):
        m = math.isqrt(n * n // 3) + 1
        perm_count = count_irreducible_components([n], n)
        det_count = count_irreducible_components([m], m)
        if perm_count <= det_count:
            return {
                "metric_name": "irreducible_components",
                "metric_value": perm_count,
                "instances_tested": 1,
                "conjecture_holds": False,
                "counterexample": f"n={n}, m={m}: {perm_count} ≤ {det_count}"
            }
    return {
        "metric_name": "irreducible_components",
        "metric_value": perm_count,
        "instances_tested": 40 - 1, # Exclude the first seed
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a65f3c06.py", line 80, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^^^

```

